Open in app ↗ Sign up Sign In

Published in Towards Data Science

Thomas Albin Follow

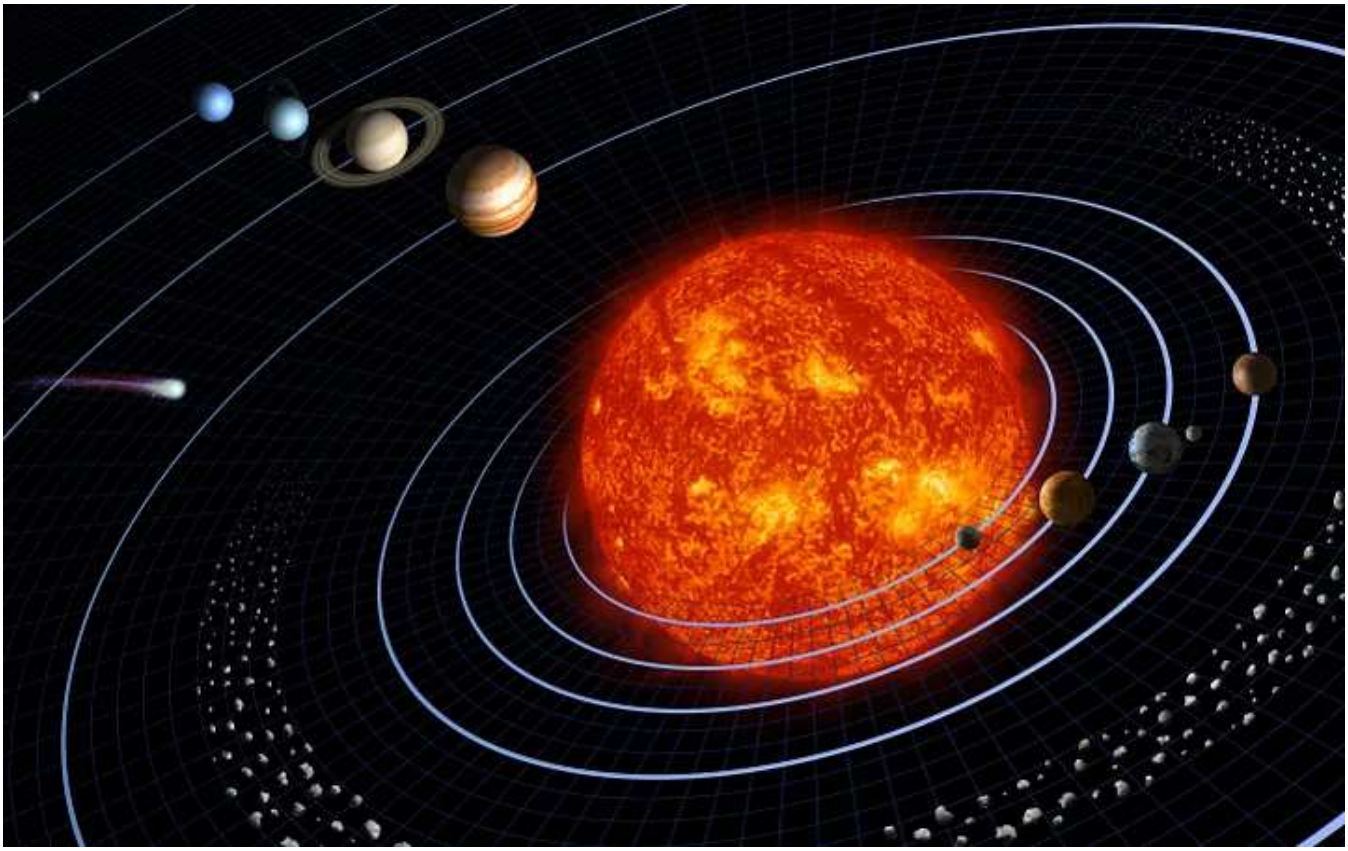
Apr 29, 2020 · 10 min read · ✦ · Listen

Save

HTTPS://TOWARDSDATASCIENCE.COM/TAGGED/SPACE-SCIENCE-WITH-PYTHON

# Space Science with Python — The Solar System centre

Third part of the Python tutorial series



Our Solar System in a schematic overview (not in scale), Credit: NASA/JPL

## Preface

17

*This is the 3rd part of my Python tutorial series "Space Science with Python". All codes that are shown here are uploaded on GitHub. Enjoy!*

**Last time ...**

... we computed and visualised the movement of the Solar System Barycentre (SSB) with respect to the Sun. A 2-dimensional projection on the Ecliptic plane revealed an impressive behaviour of the SSB: The barycentre's position leaves the Sun on a regular basis, although the Sun contains more than 99 % of the Solar System's mass! One can understand that this movement is caused by the other celestial objects, like e.g., the planets, the asteroids and even the dust. *But what are the main contributors and can we visualise it?* In this tutorial, we will aim for this scientific question and make an analysis of the results from last time.

**Let's start**

For our analysis we will use the same time interval and methods as shown last time. We will use additionally another great library: *pandas*. *pandas* is commonly used among data scientists, data engineers and also scientists. One can create so called data-frames that contain data in a tabular form. Filtering, adding columns, applying functions or SQL-similar requests allow one to manipulate and analyse data that is stored in-memory. If you did not install pandas yet, do so by using the *pip3* package manager in your virtual environment:

```
pip3 install pandas
```

If you are not familiar with *pandas*, I would recommend some tutorials, where you can learn the functionalities and advantages of this package. I will explain the shown functions, but I highly encourage you to deep dive into this data driven tool.

Our first SPICE function is again *furnsh* that loads all requested SPICE kernel files that are listed in the SPICE meta file *kernel_meta.txt*. I already downloaded and added the according kernels for this article, so you can simply pull the recent

update from GitHub. Later tutorials require large kernels that need to be downloaded manually. We will learn how to navigate through the official repository and how to choose the correct kernels in a separate tutorial.

Again, we choose a time interval starting at the 1st January 2000 and add 10,000 days. We convert the UTC time strings to the Ephemeris time (ET) with the function *utc2et* and create a *numpy* array that contains all ET time steps.

Part 1/8

*How should we proceed now with the analysis of the SSB's movement?* Well first of all, we need to compute the trajectory of the barycentre with respect to the Sun. Last time,

we did this in a for-loop. From now on, we use the *pandas* data-frames to store all necessary results in one place.

SPICE returns spatial information in km. Again, we want to scale it with the Sun's radius. We extract this information from the kernel with the function *bodvcd* (Note: SPICE knows the content of each kernel, you do not need to specify the kernel's name or location after loading it):

Part 2/8

The results will be stored in a data-frame called *SOLAR_SYSTEM_DF* that is defined empty in the beginning. Line 6 creates a column called *ET* and fills the data-frame with the corresponding ET values. *.loc* is used in *pandas* to allocate columns and is

also used to filter the data-frame. In line 12+13 we compute the date-time objects of the ETs. The *spiceypy* function *et2datetime* is used for this purpose, however, it is not a part of the official SPICE toolkit. In SPICE, the function *et2utc* is used where one can set miscellaneous date-time formats; the output itself is a string.

Part 3/8

We add and compute now three new columns. First, *POS_SSB_WRT_SUN* is computed that contains the position vector of the SSB as seen from the Sun. For this computation we need the ET column and apply row-wise (with *.apply*) the SPICE function *spkgps* via a lambda function. *spkgps* requires the target NAIF ID (*targ*), the ET (*et*), the reference frame (*ref*; here it is the Ecliptic reference frame *ECLIPJ2000*)

and the observer's NAIF ID (*obs*). The ID for the SSB and Sun is 0 and 10, respectively.

The computed position vectors are then scaled with the Sun's radius (line 11 + 12). We apply row-wise a lambda function that divides the arrays by the radius.

Finally, the distance between the Sun and the SSB is computed, using the SPICE function *vnorm*. The function computes the length (so-called norm) of a vector and is identical to the *numpy* function *numpy.linalg.norm*().

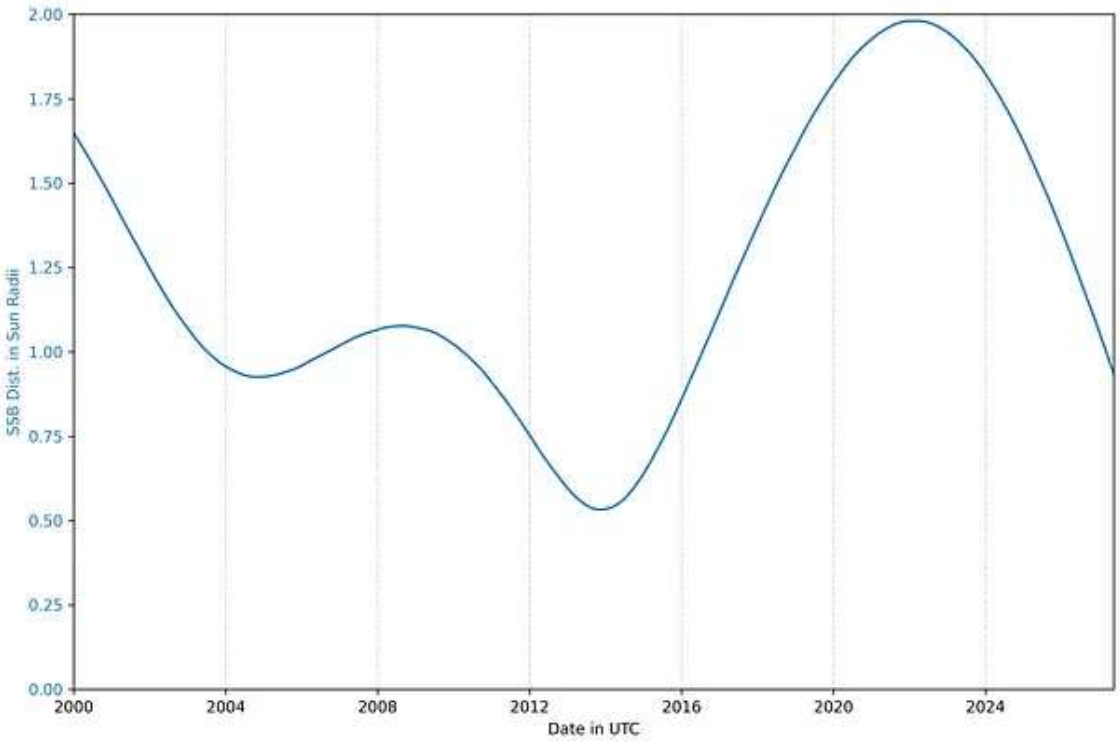
Part 4/8

Whether it is data science, space science, astronomy or any other scientific field: visualising and describing data is the first step of a proper analysis.

So let's visualise the distance between the SSB and the Sun as a function of time. For this purpose we use the module *matplotlib*, as introduced last time. We use the data of the column *SSB_WRT_SUN_SCALED_DIST* and plot it against the *UTC* date-time.

The following code creates a plot that is shown below.

Part 5/8




Distance between the Solar System Barycentre (SSB) vs. the UTC date-time. The distance is given in Solar Radii. 1 Solar Radius corresponds to 696,000 km. Credit: T. Albin (2020)

Let's describe and understand the plot. We see the distance between the SSB and the Sun vs. the date-time given in UTC. The distance-axis is scaled between 0 and 2 Solar Radii and the time-axis starts at the year 2000 and ends right before 2028. We see that the distance varies in time, however, the variation appears to be modulated (different amplitudes; and also frequencies (?)). The distance between two local maxima (around the year 2009 and 2022) is approximately a decade!

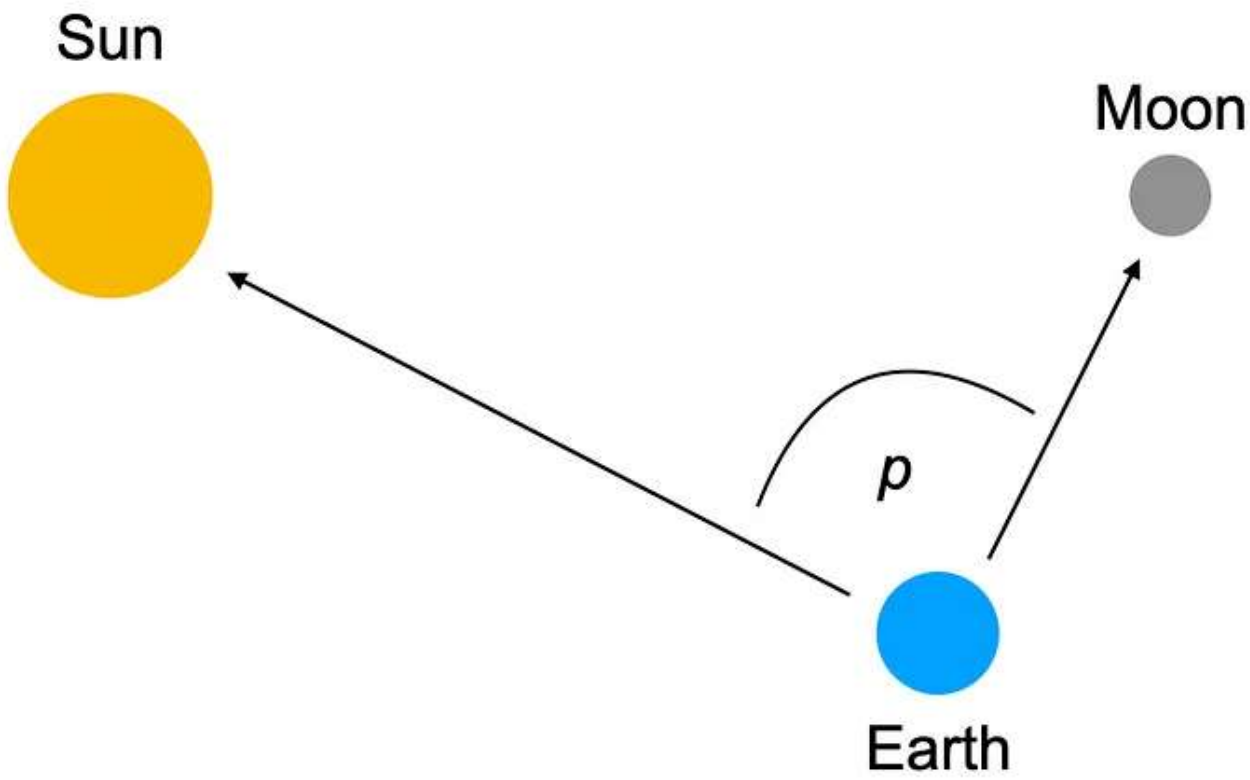
*What do these slow temporal changes mean?* The SSB's position w.r.t. the Sun can be theoretically computed by the sum of all gravity forces (planets, asteroids, dust, etc.). The inner planets (Mercury to Mars) have an orbital period between 90 days and almost 2 years. If the masses of these planets would be significant for the computation of the SSB, we would see a lot of "spikes" and fast changing variations, but we do not.

Compared to the inner planets, the outer gas giants (Jupiter, Saturn, Uranus and Neptune) have a larger mass and a larger orbital period (Jupiter revolves the Sun in around 12 years, Saturn needs almost 30 years). Although they are farer away (Jupiter around 5 AU, Saturn around 9.5 AU) it appears that they influence the SSB's position severely.

Can we simply *see* this effect in the data?

**The Phase Angle**

Today (29th April 2020) we see the Moon in its waxing phase. With every second the illuminated area as seen from the Earth increases. In a few days the Moon is fully illuminated and its waning phase begins. During its revolutions around us, the Moon appears in different so called *phase angles* w.r.t. the Sun. Consider the following sketch:




Phase angle (p) between the Moon and Sun as seen from Earth. Credit: T. Albin (2020)

The phase angle is the angle between the directional vectors of the Sun and the Moon as seen from the Earth. Generally speaking: the phase angle is the angle that is enclosed by two directional vector as seen from an observer. Thus, the phase angle can be computed between any celestial objects and is defined between 0 and 180 degrees.

*How is this related to the SSB analysis?* As mentioned before, we have the theory that the gas giants' gravitational pull is the most relevant factor of the SSB's movement. A

large distance between the SSB and the Sun's centre could mean that the outer planets "line up" to create a stronger gravitational pull from the same direction.

To check this phase angle dependency, we add new columns to our data-frame that contain the angles for all gas giants. First, we set up a dictionary that contains an abbreviated name for the planets (barycentres) with the corresponding planet's NAIF ID codes.

A for loop iterates through the dictionary and creates dynamically 2 new columns for each planet (the substring %s is replaced by the planet's abbreviation):

- *POS_%s_WRT_SUN*: Position vector of the planet as seen from the Sun. For this computation, we use again the *.apply* functionality on the ETs and use the SPICE function *spkgps* for this purpose.
- *PHASE_ANGLE_SUN_%s2SSB*: Phase angle between the planet and the SSB as seen from the Sun. *.apply* is applied on *axis=1*, since we need two columns for this computation, namely *POS_%s_WRT_SUN* and the SSB's direction vector from the Sun *POS_SSB_WRT_SUN*. The angle between these two vectors is calculated with the SPICE function *vsep*. The result is given in rad, thus the *numpy* function *numpy.degrees()* is used to convert it to degrees.

Part 6/8

*How is the angle (p) calculated between two vectors (a, b)?* The dot product between two vectors is equal to the multiplication of the length of both vectors times the cosine of the enclosed angle. We rearrange the equation to the angle and get the following equation:

$$p = \arccos \frac{\vec{a} \cdot \vec{b}}{||\vec{a}|| \cdot ||\vec{b}||}$$

Credit: T. Albin (2020)

Now, let's verify the *vsep* results. We define a lambda function where we use *numpy*'s *numpy.arccos()*, *.dot()* and *.linalg.norm()* functions. We use the very first entry of our data-frame and compute the phase angle between the SSB and Jupiter as seen from the Sun. The *pandas* function *.iloc[]* allows one to access data via an integer index; here: 0. For both functions we get a result of around 14.9°.

We have now a sophisticated data-frame that contains miscellaneous parameters. We got the time in UTC and ET, we have the distance between the Sun and the SSB as well as the phase angles for all gas giants. Now we want to plot the data to see a link between the SSB-Sun distance and the phase angles.

### The gravitational pull of the gas giants

*How shall we plot the data?* We have two different types of data we want to show. First: the distance SSB-Sun (as shown before) and the phase angle between the SSB and each planet as seen from the Sun. To see any link we need to plot them in one plot, resulting in two different y-axes (the x-axis is the time in UTC). We have the phase angles of 4 planets, resulting in 5 curves in one plot. The resulting plot would be too crowded and one could easily lose track of what is shown.

So for each planet we create an individual sub-plot. The plots are aligned vertically, to share the date-time axis. Line 3 prepares the figure and creates 4 *matplotlib* axes.

The for-loop iterates through all *matplotlib* axes and planet data, respectively. Let's have a deep dive:

- **Line 15:** First we create a title for each sub-plot. The name is the planet's name. Since each planet has its own plot we do not need to set a legend.
- **Line 18–20:** Here, we plot the SSB-Sun distance and choose a blue color to distinguish between the distance and phase angle curves.
- **Line 24–30:** Some plot formatting, where we add a y-label, axis limits, and colours.
- **Line 33:** This command tells *matplotlib* to create a twin plot in the same *matplotlib* axis. *.twinx()* indicates that the x axis (date-time in UTC) is copied.
- **Line 36–39:** Now we plot the phase angle between the planet and the SSB (Observer: the Sun). This curve is coloured in orange …
- **Line 42–49:** … as well as the corresponding y axis label and ticks (the numbers shown on the y axis).

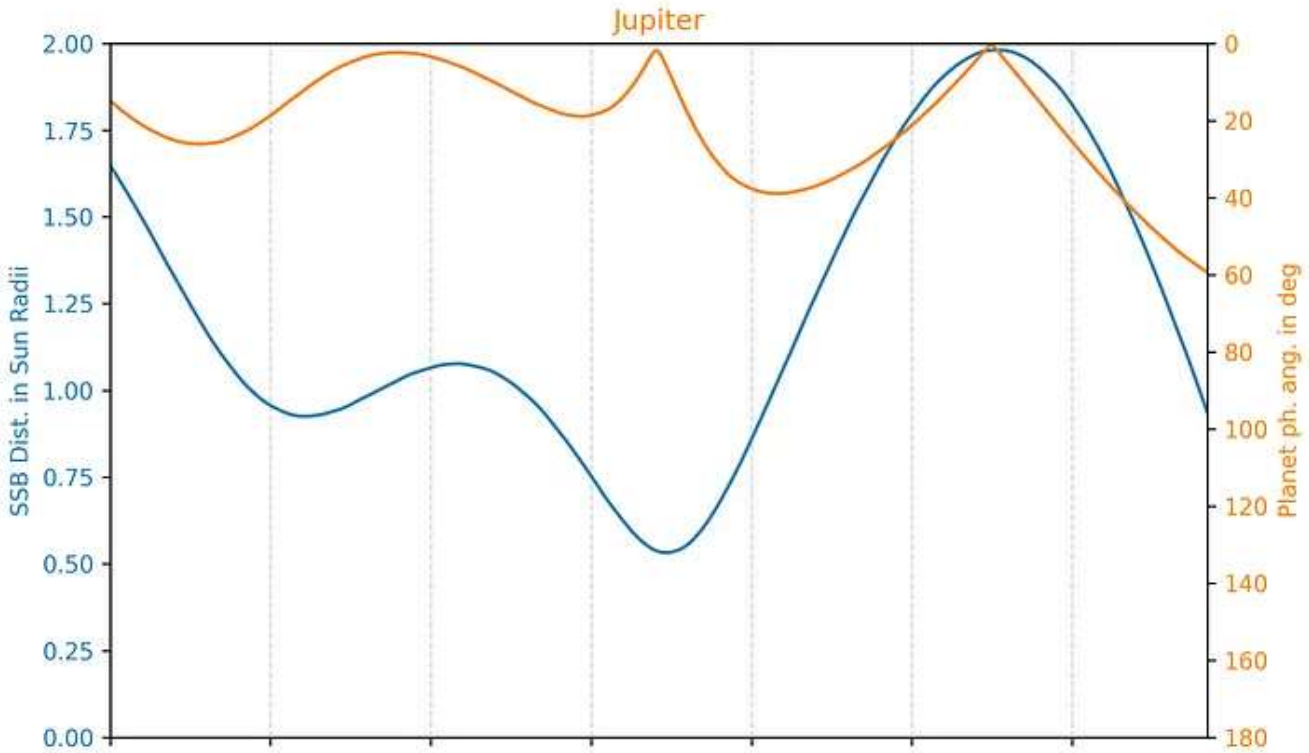
- **Line 52:** For a better readability or eye guidance we add a vertical grid along the date-time.

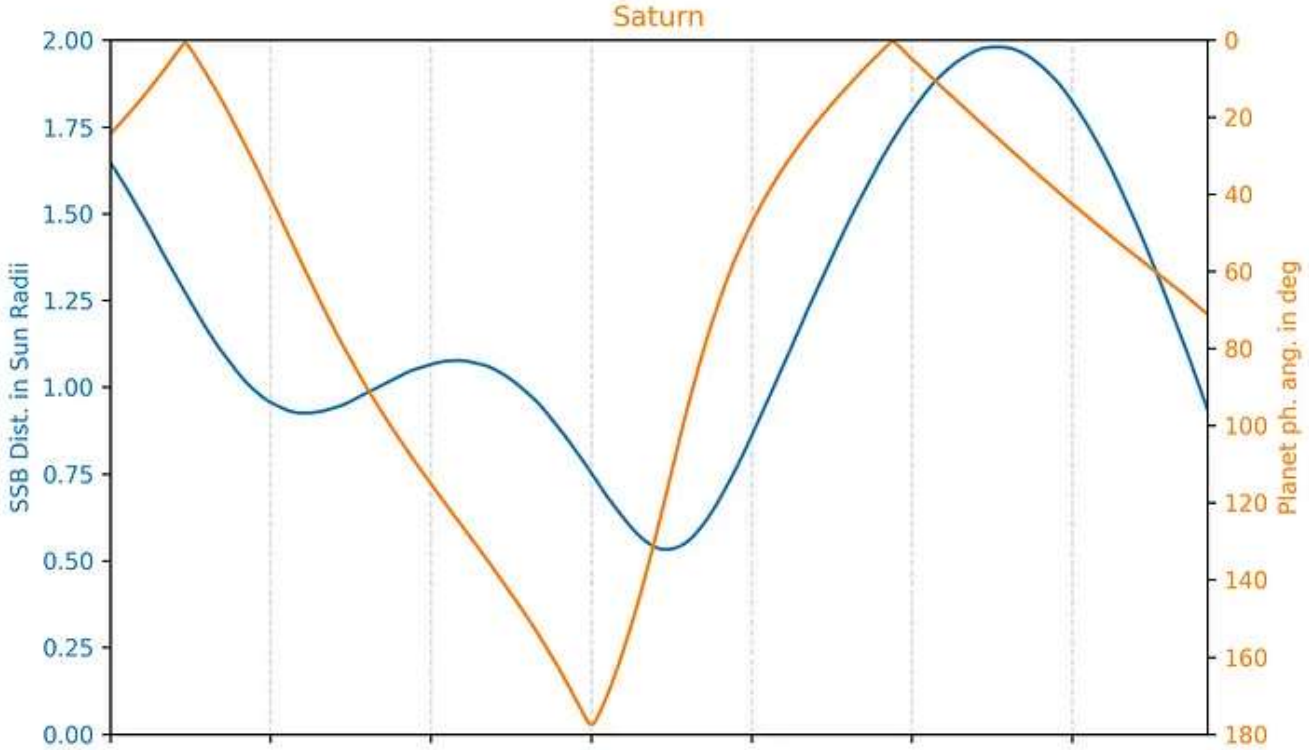
At the end we set a label for the time axis, reduce the space between the sub-plots and save the plot.

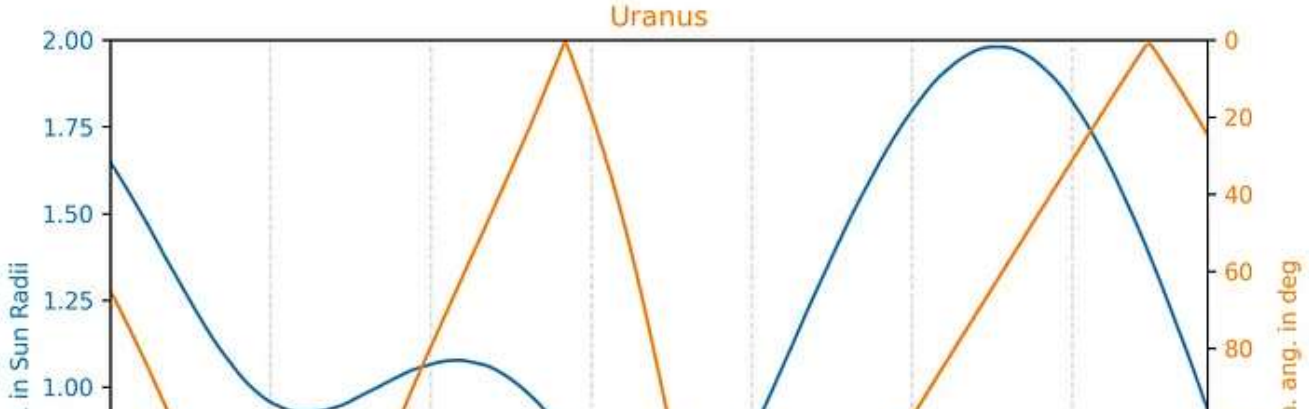
Part 8/8

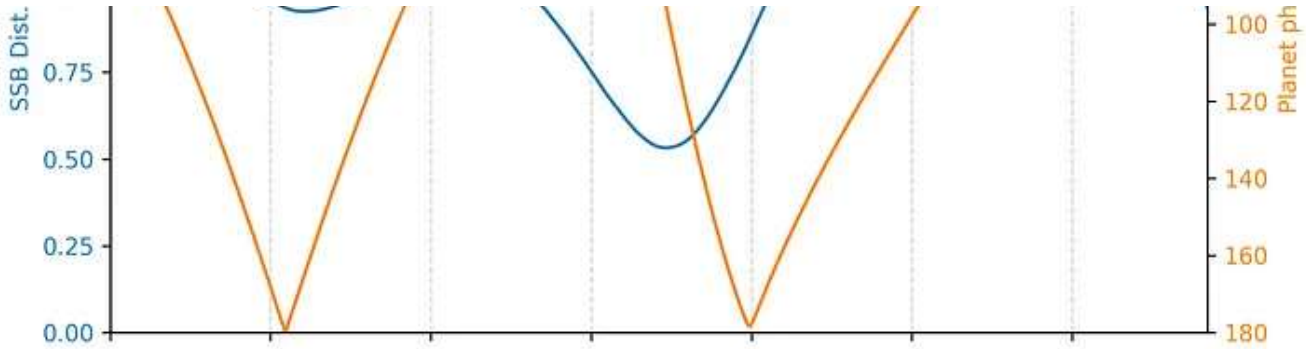
All subplots are shown below. You can see the blue and orange curves representing the SSB-Sun distance and phase angle between the SSB and planet, respectively. The title shows the name of the planet and the right y axis is inverted. As you can see, the large distance between the SSB and the Sun correlates between the years 2020 and 2024 with the small phase angle to the planet Jupiter. However, this effect is less

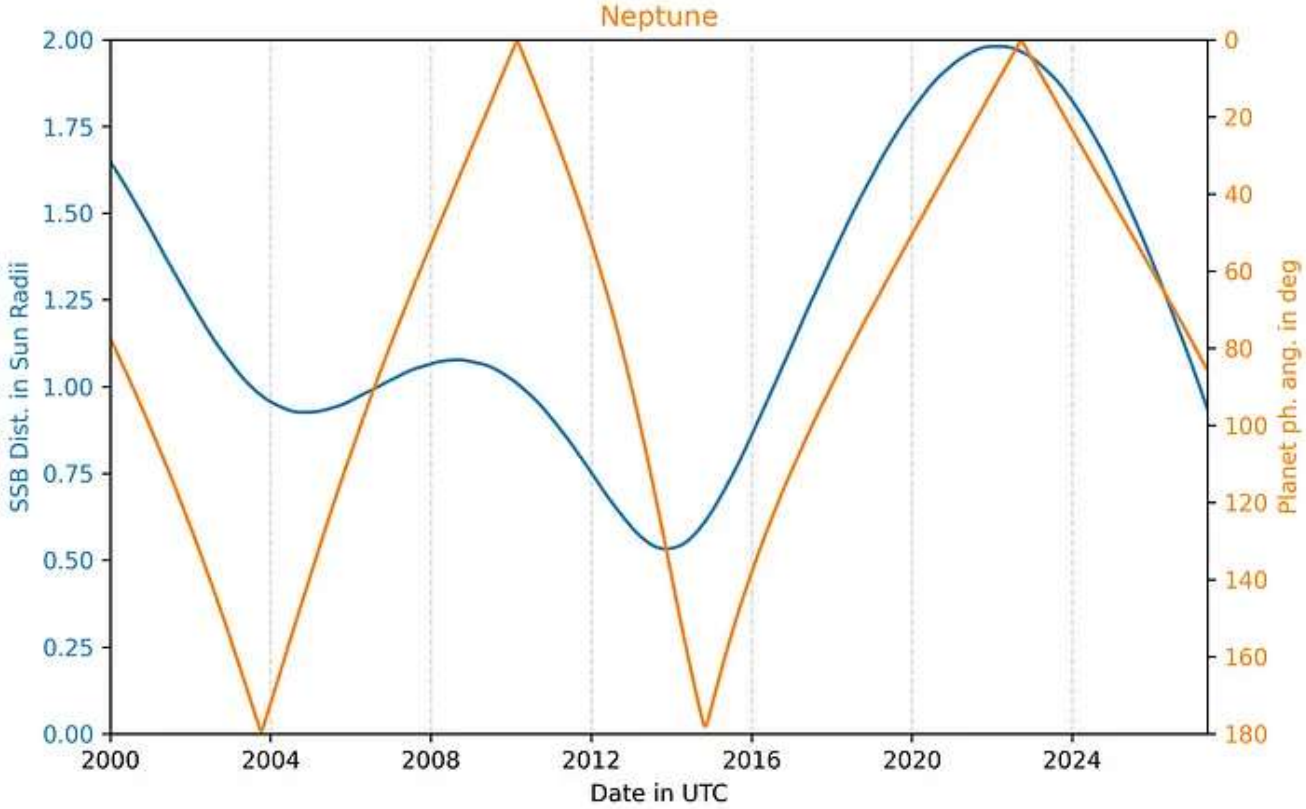
severe for the first local maximum between 2008 and 2012 and does not correlate at all with the minimum between 2012 and 2016. Let's have a look at this particular minimum. You can see that the phase angles for Saturn, Uranus and Neptune are way larger and create a "counter gravity pull" to the other direction causing the SSB's position to stay within the Sun. Between 2020 and 2024 the gas giants are more aligned to the same direction causing a maximum distance of almost 2 Solar Radii!


Jupiter
SSB Dist. in Sun Radii
Planet ph. ang. in deg
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
0
20
40
60
80
100
120
140
160
180


Saturn
SSB Dist. in Sun Radii
Planet ph. ang. in deg
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
0
20
40
60
80
100
120
140
160
180


Uranus
2.00
1.75
1.50
1.25
1.00
0
20
40
60
80




Creating plots in science is often challenging and time consuming. *Are there other ways to visualise the data and analysis?* Of course! Visualising data in a certain way is only one possible solution. If you would like to share your ideas or plot solutions, feel free to do it. Here or on GitHub.

In the meantime, stay healthy and enjoy the night sky,

Thomas

Python Space Science Programming Space Science With Python

## Sign up for The Variable

By Towards Data Science

Every Thursday, the Variable delivers the very best of Towards Data Science: from hands-on tutorials and cutting-edge research to original features you don't want to miss. Take a look.

By signing up, you will create a Medium account if you don't already have one. Review our Privacy Policy for more information about our privacy practices.

Get this newsletter

About Help Terms Privacy

Get the Medium app

Download on the App Store

GET IT ON Google Play